1.

Partimos desde la definición

$$P_{ij}^{(\alpha)} := \frac{n_\alpha}{|G|}\sum_{g\in G} D_{ij}^{(\alpha)}(g^{-1})r(g)$$

Y por tanto

$$\begin{aligned}
P_{ij}^{(\alpha)}P_{kl}^{(\beta)} &= \left(\frac{n_\alpha}{|G|}\sum_{g\in G} D_{ij}^{(\alpha)}(g^{-1})r(g)\right)\left(\frac{n_\beta}{|G|}\sum_{h\in G} D_{kl}^{(\beta)}(h^{-1})r(h)\right)\\
&= \frac{n_\alpha n_\beta}{|G|^2}\sum_{g,h\in G} D_{ij}^{(\alpha)}(g^{-1})D_{kl}^{(\beta)}(h^{-1})r(g)r(h)\\
&= \frac{n_\alpha n_\beta}{|G|^2}\sum_{g,h\in G} D_{ij}^{(\alpha)}(g^{-1})D_{kl}^{(\beta)}(h^{-1})r(gh)
\end{aligned}$$

Ahora podemos tomar una nueva variable $x = gh \implies h = g^{-1}x$ y por tanto

$$\begin{aligned}
P_{ij}^{(\alpha)}P_{kl}^{(\beta)} &= \frac{n_\alpha n_\beta}{|G|^2}\sum_{g,x\in G} D_{ij}^{(\alpha)}(g^{-1})D_{kl}^{(\beta)}((g^{-1}x)^{-1})r(x)\\
&= \frac{n_\alpha n_\beta}{|G|^2}\sum_{g,x\in G} D_{ij}^{(\alpha)}(g^{-1})D_{kl}^{(\beta)}(x^{-1}g)r(x)\\
D_{kl}^{(\beta)}(x^{-1}g) &= \sum_{m=1}^{n_\beta} D_{km}^{(\beta)}(x^{-1})D_{ml}^{(\beta)}(g)\\
P_{ij}^{(\alpha)}P_{kl}^{(\beta)} &= \frac{n_\alpha n_\beta}{|G|^2}\sum_{g,x\in G} D_{ij}^{(\alpha)}(g^{-1})\sum_{m=1}^{n_\beta} D_{km}^{(\beta)}(x^{-1})D_{ml}^{(\beta)}(g)r(x)\\
&= \frac{n_\alpha n_\beta}{|G|^2}\sum_{x\in G} r(x)\sum_{m=1}^{n_\beta} D_{km}^{(\beta)}(x^{-1})\left(\sum_{g\in G} D_{ml}^{(\beta)}(g)D_{ij}^{(\alpha)}(g^{-1})\right)\\
\sum_{g\in G} D_{ml}^{(\beta)}(g)D_{ij}^{(\alpha)}(g^{-1}) &= \frac{|G|}{n_\alpha}\delta_{\alpha\beta}\delta_{il}\delta_{mj}\\
P_{ij}^{(\alpha)}P_{kl}^{(\beta)} &= \frac{n_\alpha n_\beta}{|G|^2}\sum_{x\in G} r(x)\sum_{m=1}^{n_\beta} D_{km}^{(\beta)}(x^{-1})\left(\frac{|G|}{n_\alpha}\delta_{\alpha\beta}\delta_{il}\delta_{mj}\right)
\end{aligned}$$

Dado que tenemos δ_{mj} entonces esa sumatoria se reduce a simplemente el caso $m = j$

$$\begin{aligned}
P_{ij}^{(\alpha)}P_{kl}^{(\beta)} &= \delta_{\alpha\beta}\delta_{il}\frac{n_\beta}{|G|}\sum_{x\in G} D_{kj}^{(\beta)}(x^{-1})r(x)\\
&= \delta_{\alpha\beta}\delta_{il}P_{kj}^{(\beta)}
\end{aligned}$$

Esto ya es el resultado pues sabemos que en caso de que $\alpha \neq \beta$ entonces todo se va a 0. Por lo tanto

$$P_{ij}^{(\alpha)}P_{kl}^{(\beta)} = \delta_{\alpha\beta}\delta_{il}P_{kj}^{(\alpha)}$$

2.

2.1. $P^2 = P$.

$$\begin{aligned}
\left(P^{(\alpha)}\right)^2 &= \sum_{k,j=1}^{n_{(\alpha)}} P^{\alpha}_{kk} P^{(\alpha)}_{jj} \\
P^{(\alpha)}_{kk} P^{(\alpha)}_{jj} &= \delta_{\alpha\alpha}\delta_{kj} P^{(\alpha)}_{kj} \\
\left(P^{(\alpha)}\right)^2 &= \sum_{k,j=1}^{n_{(\alpha)}} \delta_{kj} P^{(\alpha)}_{kj} \\
\left(P^{(\alpha)}\right)^2 &= \sum_{k=1}^{n_{(\alpha)}} P^{(\alpha)}_{kk} \\
\left(P^{(\alpha)}\right)^2 &= P^{\alpha}
\end{aligned}$$

2.2. $P^* = P$.

$$\begin{aligned}
\left(P^{(\alpha)}\right)^* &= \left(\frac{n_\alpha}{|G|}\sum_{g\in G}\chi^{(\alpha)}(g^{-1})r(g)\right)^* \\
&= \frac{n_\alpha}{|G|}\sum_{g\in G}\chi^{(\alpha)}(g)r(g^{-1})
\end{aligned}$$

Pero esto es equivalente pues si tomamos $g^{-1} = h$ podemos cambiar de variable y recorrer el mismo espacio. Es decir

$$\begin{aligned}
\left(P^{(\alpha)}\right)^* &= \frac{n_\alpha}{|G|}\sum_{h\in G}\chi^{(\alpha)}(h^{-1})r(h) \\
\left(P^{(\alpha)}\right)^* &= P^{(\alpha)}
\end{aligned}$$

2.3. **Multiplicidad** m_α.

$$\begin{aligned}
tr(P^{(\alpha)}) &= tr\left(\frac{n_\alpha}{|G|}\sum_{g\in G}\chi^{(\alpha)}(g^{-1})r(g)\right) \\
&= \frac{n_\alpha}{|G|}\sum_{g\in G}\chi^{(\alpha)}(g^{-1})tr\left(r(g)\right)
\end{aligned}$$

Sabemos que la traza de r sera el caracter completo que a su vez sabemos corresponde a la suma de los caracteres irrep con sus multiplicidades. Por lo tanto,

$$tr(r(g)) = \sum_{\beta\in G} m_\beta \chi^{(\beta)}(g)$$

Por lo tanto

$$\begin{aligned}
tr(P^{(\alpha)}) &= \frac{n_\alpha}{|G|}\sum_{g\in G}\chi^{(\alpha)}(g^{-1})\sum_{\beta\in G} m_\beta\chi^{(\beta)}(g) \\
tr(P^{(\alpha)}) &= \frac{n_\alpha}{|G|}\sum_{\beta\in G} m_\beta \sum_{g\in G}\chi^{(\alpha)}(g^{-1})\chi^{(\beta)}(g)
\end{aligned}$$

Dado que los caracteres son ortogonales entre si esta suma solamente tiene sentido cuando $\alpha = \beta$

$$
\begin{aligned}
tr(P^{(\alpha)}) &= \frac{n_\alpha}{|G|} m_\alpha \sum_{g\in G} \chi^{(\alpha)}(g^{-1})\chi^{(\alpha)}(g) \\
&= \frac{n_\alpha}{|G|} m_\alpha \, |G| \\
&= n_\alpha m_\alpha \\
m_\alpha &= \frac{1}{n\alpha} tr(P^{(\alpha)})
\end{aligned}
$$

2.4. **caracteristicas $P_k k^{(\alpha)}$.** Las primeras caracteristicas ya las mostramos en las correspondientes demostraciones (es decir, utilizamos ambas cosas en los puntos de arriba). Sin embargo falta mostrar que $tr(P_{kk}^{(\alpha)}) = m_\alpha$.

$$
\begin{aligned}
tr(P_{kk}^{(\alpha)}) &= tr\left(\frac{n_\alpha}{|G|} \sum_{g\in G} D_{kk}^{(\alpha)}(g^{-1}) r(g)\right) \\
tr(r(g)) &= \sum_{\beta\in G} m_\beta \chi^{(\beta)}(g) = \sum_{\beta\in G} m_\beta \sum_{l=1}^{n_\alpha} D_{ll}^{(\beta)}(g) \\
tr(P_{kk}^{(\alpha)}) &= \frac{n_\alpha}{|G|} \sum_{g\in G} D_{kk}^{(\alpha)}(g^{-1}) \sum_{\beta\in G} m_\beta \sum_{l=1}^{n_\alpha} D_{ll}^{(\beta)}(g) \\
tr(P_{kk}^{(\alpha)}) &= \frac{n_\alpha}{|G|} \sum_{\beta\in G} m_\beta \sum_{g\in G} \sum_{l=1}^{n_\alpha} D_{kk}^{(\alpha)}(g^{-1}) D_{ll}^{(\beta)}(g) \\
tr(P_{kk}^{(\alpha)}) &= \frac{n_\alpha}{|G|} m_\alpha \sum_{g\in G} D_{kk}^{(\alpha)}(g^{-1}) D_{kk}^{(\alpha)}(g) \\
tr(P_{kk}^{(\alpha)}) &= \frac{n_\alpha}{|G|} m_\alpha \frac{|G|}{n_\alpha} \\
tr(P_{kk}^{(\alpha)}) &= m_\alpha
\end{aligned}
$$

3.

Para demostrar que $v_{kj}^{(\alpha)}$ son linealmente independientes basta con mostrar que

$$
\sum_{j=1}^{n_\alpha} c_j v_{kj}^{(\alpha)} = 0 \implies \forall j;\ c_j = 0
$$

Desarrollando esto

$$
\begin{aligned}
\sum_{j=1}^{n_\alpha} c_j v_{kj}^{(\alpha)} &= 0 \\
\sum_{j=1}^{n_\alpha} c_j P_{1j}^{(\alpha)} \omega_k &= 0
\end{aligned}
$$

Ahora, sin perdida de generalidad tomemos para un $m \in \{1, \ldots, n_\alpha\}$ un $P^{(\alpha)}_{m1}$ para aplicarlo lo que nos daria.

$$\begin{aligned}
\sum_{j=1}^{n_\alpha} c_j P^{(\alpha)}_{m1} P^{(\alpha)}_{1j} \omega_k &= P_{m1} 0 \\
\sum_{j=1}^{n_\alpha} c_j \left(\delta_{\alpha\alpha} \delta mj P_{11}\right) \omega_k &= 0 \\
c_m P_{11} \omega_k &= 0 \\
P_{11}\omega_k \neq 0 \implies c_m &= 0
\end{aligned}$$

Por lo tanto

$$\forall m;\ c_m = 0$$

4.

Para este caso, nos piden la evolución de un vector $v^{(\alpha)}_{kn}$ (con $n \in \{1, \ldots, n_\alpha\}$ y k, α fijos) respecto a un operador del grupo g dada la irrep r. Es decir, nos estan preguntando

$$r(g) v^{(\alpha)}_{kn} = r(g) P^{(\alpha)}_{1n} \omega_k =?$$

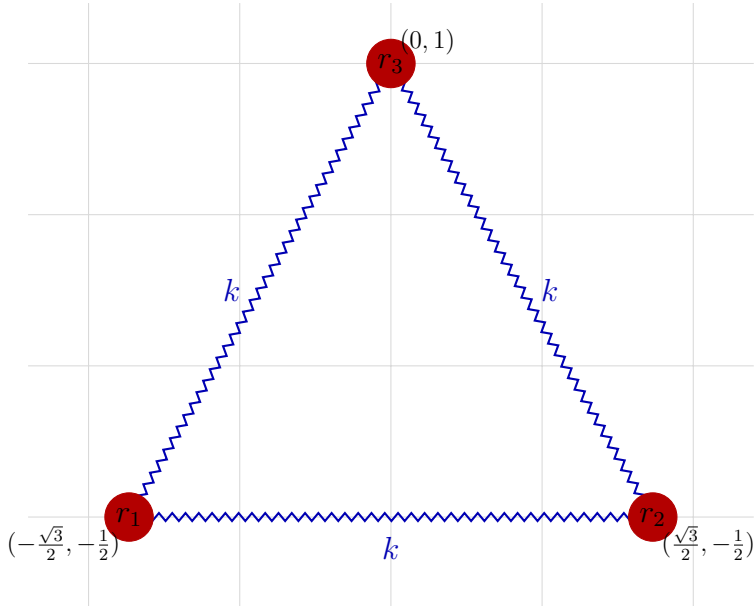
Por lo tanto podemos desarrollar

$$\begin{aligned}
r(g) v^{(\alpha)}_{kn} &= r(g) P^{(\alpha)}_{1n} \omega_k \\
&= r(g) \left(\frac{n_\alpha}{|G|} \sum_{h \in G} D^{(\alpha)}_{1j}(h^{-1}) r(h) \right) \omega_k \\
&= \frac{n_\alpha}{|G|} \sum_{h \in G} D^{(\alpha)}_{1j}(h^{-1}) r(gh) \omega_k \\
&= \frac{n_\alpha}{|G|} \sum_{x \in G} D^{(\alpha)}_{1j}(x^{-1} g) r(x) \omega_k \\
D^{(\alpha)}_{1j}(x^{-1} g) &= \sum_{m=1}^{n_\alpha} D^{(\alpha)}_{1m}(x^{-1}) D^{(\alpha)}_{mj}(g) \omega_k \\
r(g) v^{(\alpha)}_{kn} &= \frac{n_\alpha}{|G|} \sum_{x \in G} \left(\sum_{m=1}^{n_\alpha} D^{(\alpha)}_{1m}(x^{-1}) D^{(\alpha)}_{mj}(g) \right) r(x) \omega_k \\
&= \sum_{m=1}^{n_\alpha} D^{(\alpha)}_{mj}(g) \frac{n_\alpha}{|G|} \sum_{x \in G} D^{(\alpha)}_{1m}(x^{-1}) r(x) \omega_k \\
&= \sum_{m=1}^{n_\alpha} D^{(\alpha)}_{mj}(g) P^{(\alpha)}_{1m} \omega_k \\
&= \sum_{m=1}^{n_\alpha} D^{(\alpha)}_{mj}(g) v^{(\alpha)}_{kn}
\end{aligned}$$

Dado que ya mostramos que $v^{(\alpha)}_{kn}$ es una base esto demuestra que para la acción de un grupo este evoluciona respecto a aplicar la matriz $D^{(\alpha)}(g)$

5.

Iniciemos visualizando este punto:



Ahora bien, si tomamos el desplazamiento como nos pide el texto (aka $q_i(t) = (x_i, y_i)$ entonces para un tiempo t podemos saber que la posición es simplemente

$$r_i + q_i = (x_{r_i} + x_i, y_{r_i} + y_i)$$

Ahora bien, dado que estamos asumiendo que la interacción se da por medio de un resorte debemos recordar la energia potencial de un resorte que es

$$U_{resorte} = \frac{1}{2}k\Delta L^2 \tag{1}$$

con ΔL la elongación respecto al equilibrio. Es decir $\Delta L = L_{ij} - L_e$. Entonces necesitaremos calcular tanto las combinaciones de L_{ij} como L_e. Iniciemos por este ultimo, asumiendo, como se nos plantea, que con r_i esta en equilibrio entonces

$$L_e = \sqrt{(x_{r_i} - x_{r_j})^2 + (y_{r_i} - y_{r_j})^2}$$

Esto deberia dar igual para cualquier combinación de i y j. Hagamos con $i = 1$ y $j = 3$ y despues comprobemos con un script.

$$\begin{aligned}
L_e &= \sqrt{(x_{r_1} - x_{r_3})^2 + (y_{r_1} - y_{r_3})^2} \\
L_e &= \sqrt{\left(-\frac{\sqrt{3}}{2} - 0\right)^2 + (-\frac{1}{2} - 1)^2} \\
&= \sqrt{\frac{3}{4} + \frac{9}{4}} \\
&= \sqrt{\frac{12}{4}} \\
&= \sqrt{3}
\end{aligned}$$

Ahora para confirmar rapidamente tenemos el codigo podemos escribir el siguiente codigo en python

```python
from typing import Tuple
import sympy as sp

Point = Tuple[sp.Symbol, sp.Symbol]
```

```python

def distancia(p1: Point, p2: Point) -> sp.Expr:
    return sp.sqrt((p1[0] - p2[0])**2 + (p1[1] - p2[1])**2)

R_1 = (-sp.sqrt(3)/2, -sp.Rational(1,2))
R_2 = ( sp.sqrt(3)/2, -sp.Rational(1,2))
R_3 = (0, 1)

puntos = {k: v for k, v in globals().items() if 'R_' in k}

for n_i, v_i in puntos.items():
    for n_j, v_j in puntos.items():
        dist = distancia(v_i, v_j)
        dist_simplificada = sp.simplify(dist)
        print(f"Longitud Equilibrio {n_i} - {n_j}: {dist_simplificada}")
```

Que nos da como resultado

```
  Longitud Equilibrio R_1 - R_1: 0
  Longitud Equilibrio R_1 - R_2: sqrt(3)
  Longitud Equilibrio R_1 - R_3: sqrt(3)
  Longitud Equilibrio R_2 - R_1: sqrt(3)
  Longitud Equilibrio R_2 - R_2: 0
  Longitud Equilibrio R_2 - R_3: sqrt(3)
  Longitud Equilibrio R_3 - R_1: sqrt(3)
  Longitud Equilibrio R_3 - R_2: sqrt(3)
  Longitud Equilibrio R_3 - R_3: 0

```

Que nos confirma ser igual para todos los casos (que ya habiamos dicho que era lo que esperabamos). Por lo tanto $L_e = \sqrt{3}$.
Ahora para calcular ΔL tambien necesitamos L_{ij}. Para este caso podemos hacerlo de manera equivalente (con la nueva posición que definimos). A forma de ejemplo realicemos esto para $i = 1$ y $j = 3$

$$
\begin{aligned}
L_{13} &= \sqrt{\left(\left(-\frac{\sqrt{3}}{2} + x_1\right) - x_3\right)^2 + \left(\left(-\frac{1}{2} + y_1\right) - (1 + y_3)\right)^2} \\
&= \sqrt{\left(-\frac{\sqrt{3}}{2} + x_1 - x_3\right)^2 + (-1)^2\left(\frac{3}{2} - y_1 + y_3\right)^2} \\
&= \sqrt{\frac{\left(-\sqrt{3} + 2x_1 - 2x_3\right)^2 + \left(3 + 2y_3 - 2y_1\right)^2}{4}} \\
&= \frac{\sqrt{\left(-\sqrt{3} + 2x_1 - 2x_3\right)^2 + \left(3 + 2y_3 - 2y_1\right)^2}}{2}
\end{aligned}
$$

Podriamos hacer esto para cada combinación. Sin embargo, considero que lo mas facil es simplemente hacer un codigo que haga todas las combinaciones.

```python
from typing import Tuple
import sympy as sp

Point = Tuple[sp.Symbol, sp.Symbol]

x1, x2, x3, y1, y2, y3 = sp.symbols("x_1 x_2 x_3 y_1 y_2 y_3")
```

```python

R_1 = (-sp.sqrt(3)/2, -sp.Rational(1,2))
R_2 = ( sp.sqrt(3)/2, -sp.Rational(1,2))
R_3 = (0, 1)

def distancia(p1: Point, p2: Point) -> sp.Expr:
    return sp.sqrt((p1[0] - p2[0])**2 + (p1[1] - p2[1])**2)

def avanzar_tiempo(atomo: str = '1'):
    pos_equiv = globals()[f"R_{atomo}"]
    plus_x = globals()[f"x{atomo}"]
    plus_y = globals()[f"y{atomo}"]
    return (pos_equiv[0] + plus_x, pos_equiv[1] + plus_y)

puntos = ['1', '2', '3']

print('\\begin{itemize}')
for i in puntos:
    for j in puntos:
        if j >= i:
            continue
        new_r_i = avanzar_tiempo(i)
        new_r_j = avanzar_tiempo(j)
        dist = distancia(new_r_i, new_r_j)
        dist_simplificada = sp.simplify(dist)
        print(f"\t\\item $L_{{{j}{i}}}:\\ {sp.latex(dist_simplificada)}$")
print('\\end{itemize}')
```

que da como resultado

- $L_{12}:\ \sqrt{(y_1 - y_2)^2 + \left(-x_1 + x_2 + \sqrt{3}\right)^2}$
- $L_{13}:\ \frac{\sqrt{\left(-2x_1+2x_3+\sqrt{3}\right)^2+(-2y_1+2y_3+3)^2}}{2}$
- $L_{23}:\ \frac{\sqrt{\left(2x_2-2x_3+\sqrt{3}\right)^2+(-2y_2+2y_3+3)^2}}{2}$

Ahora bien, dado que necesitamos U lo unico que requerimos es sumar lo que aporta cada uno de estos componentes. Para esto lo que podemos hacer es extender el script anterior para obtener U considerando la ecuación 1.

```python
from typing import Tuple
import sympy as sp

Point = Tuple[sp.Symbol, sp.Symbol]

x1, x2, x3, y1, y2, y3 = sp.symbols("x_1 x_2 x_3 y_1 y_2 y_3")
k = sp.Symbol('k')

R_1 = (-sp.sqrt(3)/2, -sp.Rational(1,2))
R_2 = ( sp.sqrt(3)/2, -sp.Rational(1,2))
R_3 = (0, 1)

def distancia(p1: Point, p2: Point) -> sp.Expr:
    return sp.sqrt((p1[0] - p2[0])**2 + (p1[1] - p2[1])**2)

def avanzar_tiempo(atomo: str = '1'):
    pos_equiv = globals()[f"R_{atomo}"]
    plus_x = globals()[f"x{atomo}"]
```

```
    plus_y = globals()[f"y{atomo}"]
    return (pos_equiv[0] + plus_x, pos_equiv[1] + plus_y)

def U_resorte(l_new):
    return sp.Rational(1, 2) * k * (l_new - sp.sqrt(3))**2

puntos = ['1', '2', '3']

U: sp.Expr = 0
for i in puntos:
    for j in puntos:
        if j >= i:
            continue
        new_r_i = avanzar_tiempo(i)
        new_r_j = avanzar_tiempo(j)
        dist = distancia(new_r_i, new_r_j)
        dist_simplificada = sp.simplify(dist)
        U += U_resorte(dist_simplificada)

U_simpl = sp.simplify(U)
print(f"$U = {sp.latex(U_simpl)}$")
```

Y este script tiene como resultado

$$
\begin{aligned}
U = \frac{k}{8}\Bigg[& 4\left(\sqrt{(y_1-y_2)^2+(-x_1+x_2+\sqrt{3})^2}-\sqrt{3}\right)^2 \\
& +\left(\sqrt{(-2x_1+2x_3+\sqrt{3})^2+(-2y_1+2y_3+3)^2}-2\sqrt{3}\right)^2 \\
& +\left(\sqrt{(2x_2-2x_3+\sqrt{3})^2+(-2y_2+2y_3+3)^2}-2\sqrt{3}\right)^2\Bigg]
\end{aligned} \tag{2}
$$

Nota: Durante el transcurso de este trabajo si construyo sobre las bases de un codigo anterior prefiero desarrollarlo todo como archivos independientes. Esto es para que el lector pueda entender cada codigo sin necesidad de revisar otros listings. En caso de que prefiera una versión mas optimizada donde las cosas se definen una unica vez estare atento a los comentarios para mejorarlo.

6.

Aprovechando el codigo que usamos para la sección anterior podemos extenderlo para obtener el resultado. Basicamente lo unico que necesitamos es encontrar la matriz que sigue la aproximación que nos recomienda el enunciado y multiplicarla por el inverso de la matriz de masa que es simplemente $M = m\mathbb{I}$. Esto entonces se ve de la siguiente manera en codigo

```
from typing import Tuple
import sympy as sp

Point = Tuple[sp.Symbol, sp.Symbol]

x1, x2, x3, y1, y2, y3 = sp.symbols("x_1 x_2 x_3 y_1 y_2 y_3")
k = sp.Symbol('k')

R_1 = (-sp.sqrt(3)/2, -sp.Rational(1,2))
R_2 = ( sp.sqrt(3)/2, -sp.Rational(1,2))
R_3 = (0, 1)
```

```python

def distancia(p1: Point, p2: Point) -> sp.Expr:
    return sp.sqrt((p1[0] - p2[0])**2 + (p1[1] - p2[1])**2)

def avanzar_tiempo(atomo: str = '1'):
    pos_equiv = globals()[f"R_{atomo}"]
    plus_x = globals()[f"x{atomo}"]
    plus_y = globals()[f"y{atomo}"]
    return (pos_equiv[0] + plus_x, pos_equiv[1] + plus_y)

def U_resorte(l_new):
    return sp.Rational(1, 2) * k * (l_new - sp.sqrt(3))**2

### Nuevo ################
def calcular_matriz(U, derivandos):
    def doble_derivada(U, i, j):
        return sp.diff(sp.diff(U, i), j)

    n = len(derivandos)

    matrix = [[sp.Integer(0) for _ in range(n)] for _ in range(n)]

    for n_i, i in enumerate(derivandos):
        for n_j, j in enumerate(derivandos):
            derivada_segunda = doble_derivada(U, i, j)
            valor_equilibrio = derivada_segunda.subs({var: 0 for var in derivandos})
            matrix[n_i][n_j] = valor_equilibrio
    return sp.Matrix(matrix)
###################


puntos = ['1', '2', '3']

U: sp.Expr = 0
for i in puntos:
    for j in puntos:
        if j >= i:
            continue
        new_r_i = avanzar_tiempo(i)
        new_r_j = avanzar_tiempo(j)
        dist = distancia(new_r_i, new_r_j)
        dist_simplificada = sp.simplify(dist)
        U += U_resorte(dist_simplificada)

U_simpl = sp.simplify(U)

### Nuevo ################
m = sp.Symbol('m')

h = [x1, y1, x2, y2, x3, y3]

K = calcular_matriz(U, h)

M = m * sp.eye(len(h))

```

```
Lambda = M.inv() * K

print("\nMatriz Lambda:")
print(f"\\begin{{equation}}\n {sp.latex(Lambda)} \n\\end{{equation}}")
###################
```

que da como resultado:
Matriz Lambda:

$$\begin{bmatrix} \frac{5k}{4m} & \frac{\sqrt{3}k}{4m} & -\frac{k}{m} & 0 & -\frac{k}{4m} & -\frac{\sqrt{3}k}{4m} \\ \frac{\sqrt{3}k}{4m} & \frac{3k}{4m} & 0 & 0 & -\frac{\sqrt{3}k}{4m} & -\frac{3k}{4m} \\ -\frac{k}{m} & 0 & \frac{5k}{4m} & -\frac{\sqrt{3}k}{4m} & -\frac{k}{4m} & \frac{\sqrt{3}k}{4m} \\ 0 & 0 & -\frac{\sqrt{3}k}{4m} & \frac{3k}{4m} & \frac{\sqrt{3}k}{4m} & -\frac{3k}{4m} \\ -\frac{k}{4m} & -\frac{\sqrt{3}k}{4m} & -\frac{k}{4m} & \frac{\sqrt{3}k}{4m} & \frac{k}{2m} & 0 \\ -\frac{\sqrt{3}k}{4m} & -\frac{3k}{4m} & \frac{\sqrt{3}k}{4m} & -\frac{3k}{4m} & 0 & \frac{3k}{2m} \end{bmatrix} \tag{3}$$

Nota: Dado que el problema que tiene mantener cada archivo por separado es que es dificil encontrar los cambios relevantes decidi marcar lo que agregue al script con comentarios (que me parecen evidentes). Quedo muy pendiente de si se considera mejor la otra opción. Tambien pense en la posibilidad de adjuntarlo como se haria en un jupyter notebook (es decir, por celdas) pero considere que, al menos por ahora, lo mejor era este metodo.

7.

Dado que vamos a trabajar con matrices 6×6 pero en bloques de 2×2 creo que es valioso expresar que lo que haremos es, en esencia tomar

$$\begin{pmatrix} x_1 \\ y_1 \\ x_2 \\ y_2 \\ x_3 \\ y_3 \end{pmatrix} \to \begin{pmatrix} q_1 \\ q_2 \\ q_3 \end{pmatrix}$$

Como estamos realizando esta división en atomos podemos entonces factorizar cada matriz como un producto tensorial entre el espacio de permutaciones entre los atomos y una transformación geometrica de los vectores locales. Es decir que podemos expresar la matriz como

$$r(g) = P \otimes D^{(2)}(g)$$

Donde P es la permutación de sitios y $D^{(2)}(g)$ es la representación vectorial definida en el enunciado.
Ahora bien, lo que esto significa es que podemos encontrar como varian en el espacio las simetrias los atomos entre si (es decir, como cambian de posicion los atomos) y multiplicarlo tensorialmente por el cambio que sentiria cada atomo por aparte.
Ahora bien, esto aun requiere que entendamos lo que hace cada una de las operaciones:

Nombre	Descripción	Permutaciones
e	Identidad, no cambia nada	$(q_1, q_2, q_3) \to (q_1, q_2, q_3)$
ρ	Rotación por 120°	$(q_1, q_2, q_3) \to (q_2, q_3, q_1)$
τ	Rotación por 240°	$(q_1, q_2, q_3) \to (q_3, q_1, q_2)$
σ_1	Reflexión respecto a r_1	$(q_1, q_2, q_3) \to (q_1, q_3, q_2)$
σ_2	Reflexión respecto a r_2	$(q_1, q_2, q_3) \to (q_3, q_2, q_1)$
σ_3	Reflexión respecto a r_3	$(q_1, q_2, q_3) \to (q_2, q_1, q_3)$

TABLE 1. Tabla de las operaciones junto con las permutaciones que hacen, en esta tabla no consideramos el efecto dentro del propio atomo pues estamos trabajando desde la abstracción de tomar q_i completo

Ahora ya con esto, podemos desarrollar cada una de las matrices

$$r(e) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \otimes D^{(2)}(e)$$

$$r(\rho) = \begin{pmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix} \otimes D^{(2)}(\rho)$$

$$r(\tau) = \begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{pmatrix} \otimes D^{(2)}(\tau)$$

$$r(\sigma_1) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix} \otimes D^{(2)}(\sigma_1)$$

$$r(\sigma_2) = \begin{pmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{pmatrix} \otimes D^{(2)}(\sigma_2)$$

$$r(\sigma_3) = \begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix} \otimes D^{(2)}(\sigma_3)$$

8.

Aprovechando la estructura de producto tensorial $r(g) = P(g) \otimes D^{(2)}(g)$ del punto anterior, podemos calcular el carácter de la representación total como el producto de los caracteres de la representación de permutación y la representación vectorial:

$$\chi^r(g) = \chi_{perm}(g) \cdot \chi_{vec}(g)$$

Podemos dividir por clases de conjugación basandonos en la geometria del triangulo con el que estamos trabajando (y con pistas de la tabla anterior). Sin analizamos en estas clases tenemos:

- **Identidad (e):** Los 3 átomos quedan fijos ($\chi_{perm} = 3$) y la traza del desplazamiento local es 2. Total: $\chi^r(e) = 3 \times 2 = 6$.
- **Rotaciones (ρ, τ):** Ningún átomo queda fijo ($\chi_{perm} = 0$). Por tanto, $\chi^r(rot) = 0 \times (-1) = 0$.
- **Reflexiones (σ_i):** Solo el átomo sobre el eje queda fijo ($\chi_{perm} = 1$). La traza de una reflexión local en 2D es 0. Total: $\chi^r(refl) = 1 \times 0 = 0$.

Obtenemos así el vector de caracteres $\chi^r = (6, 0, 0)$. Para encontrar la descomposición en irreps $V^r = \bigoplus m_\alpha V^{(\alpha)}$, aplicamos la fórmula de reducción $m_\alpha = \frac{1}{|G|}\sum \chi^{(\alpha)}(g)^* \chi^r(g)$:

$$m_1 = \frac{1}{6}[1(6)(1) + 2(0)(1) + 3(0)(1)] = 1$$
$$m_2 = \frac{1}{6}[1(6)(2) + 2(0)(-1) + 3(0)(0)] = 2$$
$$m_3 = \frac{1}{6}[1(6)(1) + 2(0)(1) + 3(0)(-1)] = 1$$

Por lo tanto, la descomposición es: $V^r = V^{(1)} \oplus 2V^{(2)} \oplus V^{(3)}$.
Nota: Puede ver en los anexos los calculos del caracter para la representación vectorial de cada caso. No quise ponerlos aqui por que no me resultaba facil de acomodarlo sin hacerlo mas confuso.

9.

Para esto recordemos que

$$P^{(\alpha)} = \frac{n_\alpha}{|G|}\sum_{g\in G} \chi^{(\alpha)}(g)^* r(g) \tag{4}$$

por lo tanto

$$P^1 = \frac{n_\alpha}{|G|}\sum_{g\in G} \chi^{1}(g)^* r(g)$$
$$P^1 = \frac{1}{6}\sum_{g\in G} r(g)$$

Es decir, vamos a sumar cada matriz y luego dividirla entre 6. Podemos hacerlo a mano sin embargo considero bastante mas prudente hacerlo con codigo. Para esto entonces queda

```
import sympy as sp

sqrt3 = sp.sqrt(3)
un_medio = sp.S(1)/2

D_e = sp.eye(2)
D_rho = sp.Matrix([[-un_medio, -sqrt3/2], [sqrt3/2, -un_medio]])
D_tau = sp.Matrix([[-un_medio, sqrt3/2], [-sqrt3/2, -un_medio]])

D_s3 = sp.Matrix([[-1, 0], [0, 1]])
D_s1 = D_rho * D_s3 * D_tau
D_s2 = D_tau * D_s3 * D_rho

P_e = sp.eye(3)
P_rho = sp.Matrix([[0, 0, 1], [1, 0, 0], [0, 1, 0]])
P_tau = sp.Matrix([[0, 1, 0], [0, 0, 1], [1, 0, 0]])

P_s1 = sp.Matrix([[1, 0, 0], [0, 0, 1], [0, 1, 0]])
P_s2 = sp.Matrix([[0, 0, 1], [0, 1, 0], [1, 0, 0]])
P_s3 = sp.Matrix([[0, 1, 0], [1, 0, 0], [0, 0, 1]])

operaciones = [
    sp.kronecker_product(P_e, D_e),
    sp.kronecker_product(P_rho, D_rho),
    sp.kronecker_product(P_tau, D_tau),
```

```
    sp.kronecker_product(P_s1, D_s1),
    sp.kronecker_product(P_s2, D_s2),
    sp.kronecker_product(P_s3, D_s3)
]

P1 = (sp.S(1)/6) * sum(operaciones, sp.zeros(6, 6))

P1_clean = sp.nsimplify(P1)

print("Proyector $P^{(1)}$ (Exacto):")
print(f"$${sp.latex(P1_clean)}$$")
print(f"\nTraza de $P^{(1)}: {sp.simplify(P1.trace())}$")
```

que da como resultado
Proyector $P^{(1)}$ (Exacto):

$$\begin{bmatrix} \frac{1}{4} & \frac{\sqrt{3}}{12} & -\frac{1}{4} & \frac{\sqrt{3}}{12} & 0 & -\frac{\sqrt{3}}{6} \\ \frac{\sqrt{3}}{12} & \frac{1}{12} & -\frac{\sqrt{3}}{12} & \frac{1}{12} & 0 & -\frac{1}{6} \\ -\frac{1}{4} & -\frac{\sqrt{3}}{12} & \frac{1}{4} & -\frac{\sqrt{3}}{12} & 0 & \frac{\sqrt{3}}{6} \\ \frac{\sqrt{3}}{12} & \frac{1}{12} & -\frac{\sqrt{3}}{12} & \frac{1}{12} & 0 & -\frac{1}{6} \\ 0 & 0 & 0 & 0 & 0 & 0 \\ -\frac{\sqrt{3}}{6} & -\frac{1}{6} & \frac{\sqrt{3}}{6} & -\frac{1}{6} & 0 & \frac{1}{3} \end{bmatrix}$$

Traza de P^1 : 1
Ahora bien, dado que ya sabemos que la dimensión es 1 entonces cualquier vector columna de la matriz genera el espacio. Esto lo podemos entonces tomar como

$$P^1 \cdot \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \end{pmatrix}$$

Y ahora desarrollar para obtener el eigenvalue. Esto tambien lo haremos con un codigo en donde queda.

```
from typing import Tuple
import sympy as sp

Point = Tuple[sp.Symbol, sp.Symbol]

x1, x2, x3, y1, y2, y3 = sp.symbols("x_1 x_2 x_3 y_1 y_2 y_3")
k = sp.Symbol('k')

R_1 = (-sp.sqrt(3)/2, -sp.Rational(1,2))
R_2 = ( sp.sqrt(3)/2, -sp.Rational(1,2))
R_3 = (0, 1)

def distancia(p1: Point, p2: Point) -> sp.Expr:
    return sp.sqrt((p1[0] - p2[0])**2 + (p1[1] - p2[1])**2)

def avanzar_tiempo(atomo: str = '1'):
    pos_equiv = globals()[f"R_{atomo}"]
    plus_x = globals()[f"x{atomo}"]
    plus_y = globals()[f"y{atomo}"]
    return (pos_equiv[0] + plus_x, pos_equiv[1] + plus_y)
```

```python

def U_resorte(l_new):
    return sp.Rational(1, 2) * k * (l_new - sp.sqrt(3))**2

def calcular_matriz(U, derivandos):
    def doble_derivada(U, i, j):
        return sp.diff(sp.diff(U, i), j)

    n = len(derivandos)

    matrix = [[sp.Integer(0) for _ in range(n)] for _ in range(n)]

    for n_i, i in enumerate(derivandos):
        for n_j, j in enumerate(derivandos):
            derivada_segunda = doble_derivada(U, i, j)
            valor_equilibrio = derivada_segunda.subs({var: 0 for var in derivandos})
            matrix[n_i][n_j] = valor_equilibrio
    return sp.Matrix(matrix)

puntos = ['1', '2', '3']

U: sp.Expr = 0
for i in puntos:
    for j in puntos:
        if j >= i:
            continue
        new_r_i = avanzar_tiempo(i)
        new_r_j = avanzar_tiempo(j)
        dist = distancia(new_r_i, new_r_j)
        dist_simplificada = sp.simplify(dist)
        U += U_resorte(dist_simplificada)

U_simpl = sp.simplify(U)

m = sp.Symbol('m')

h = [x1, y1, x2, y2, x3, y3]

K = calcular_matriz(U, h)

M = m * sp.eye(len(h))

Lambda = M.inv() * K

sqrt3 = sp.sqrt(3)

D_e = sp.eye(2)
D_rho = sp.Matrix([[-sp.S(1)/2, -sqrt3/2], [sqrt3/2, -sp.S(1)/2]])
D_tau = sp.Matrix([[-sp.S(1)/2, sqrt3/2], [-sqrt3/2, -sp.S(1)/2]])

D_s3 = sp.Matrix([[-1, 0], [0, 1]])
D_s1 = D_rho * D_s3 * D_tau
D_s2 = D_tau * D_s3 * D_rho

P_e = sp.eye(3)
```

```python
P_rho = sp.Matrix([[0, 0, 1], [1, 0, 0], [0, 1, 0]])
P_tau = sp.Matrix([[0, 1, 0], [0, 0, 1], [1, 0, 0]])

P_s1 = sp.Matrix([[1, 0, 0], [0, 0, 1], [0, 1, 0]])
P_s2 = sp.Matrix([[0, 0, 1], [0, 1, 0], [1, 0, 0]])
P_s3 = sp.Matrix([[0, 1, 0], [1, 0, 0], [0, 0, 1]])

operaciones = [
    sp.kronecker_product(P_e, D_e),
    sp.kronecker_product(P_rho, D_rho),
    sp.kronecker_product(P_tau, D_tau),
    sp.kronecker_product(P_s1, D_s1),
    sp.kronecker_product(P_s2, D_s2),
    sp.kronecker_product(P_s3, D_s3)
]

P1 = (sp.S(1)/6) * sum(operaciones, sp.zeros(6, 6))

P1_clean = sp.nsimplify(P1)

v_1 = sp.simplify(P1_clean * sp.Matrix([0, 0, 0, 0, 0, 1]))

print(f"Vector Propio:")
print(f"$$v_1 = {sp.latex(v_1)}$$")

Lambda_v_1 = sp.simplify(Lambda * v_1)
print(f"Aplicando Lambda al Vector Propio:")
print(f"$$\\Lambda v_1 = {sp.latex(Lambda_v_1)}$$")

idx = next(i for i, val in enumerate(v_1) if val != 0)
lambda_1 = sp.simplify(Lambda_v_1[idx] / v_1[idx])

print(f"\nlambda obtenido: ${sp.latex(lambda_1)}$", end='\n\n')

diff = sp.simplify(Lambda_v_1 - (lambda_1 * v_1))
verification = diff == sp.zeros(6, 1)

print(f"Es vector propio real: {verification}")
if not verification:
    sp.pprint(diff)
```

Vector Propio:

$$v_1 = \begin{bmatrix} -\frac{\sqrt{3}}{6} \\ -\frac{1}{6} \\ \frac{\sqrt{3}}{6} \\ -\frac{1}{6} \\ 0 \\ \frac{1}{3} \end{bmatrix}$$

Aplicando Lambda al Vector Propio:

$$\Lambda v_1 = \begin{bmatrix} -\frac{\sqrt{3}k}{2m} \\ -\frac{k}{2m} \\ \frac{\sqrt{3}k}{2m} \\ -\frac{k}{2m} \\ 0 \\ \frac{k}{m} \end{bmatrix}$$

lambda obtenido: $\frac{3k}{m}$
Es vector propio real: True
Esto implica que la frecuencia de vibración es:

$$\omega_1 = \sqrt{\frac{3k}{m}}$$

10.

Para iniciar, el vector de traslación mas rigido posible significa que toda la molecula se mueve de manera conjunta (es decir $x_i = a$ y $y_i = b$ para todo i). Por lo tanto este seria un vector

$$q = \begin{pmatrix} a \\ b \\ a \\ b \\ a \\ b \end{pmatrix}$$

Que se puede ver inmediatamente que corresponde con un vector generado por

$$T_x = \begin{pmatrix} 1 \\ 0 \\ 1 \\ 0 \\ 1 \\ 0 \end{pmatrix} \; ; \; T_y = \begin{pmatrix} 0 \\ 1 \\ 0 \\ 1 \\ 0 \\ 1 \end{pmatrix}$$

Ahora bien, ninguno de estos dos esta normalizado (aunque si son ortogonales entre si) por tanto lo primero seria normalizar lo que queda

$$u_x = \frac{1}{\sqrt{3}} \begin{pmatrix} 1 \\ 0 \\ 1 \\ 0 \\ 1 \\ 0 \end{pmatrix} \; ; \; u_y = \frac{1}{\sqrt{3}} \begin{pmatrix} 0 \\ 1 \\ 0 \\ 1 \\ 0 \\ 1 \end{pmatrix}$$

Ademas, el operador E_{trasl} corresponde con

$$E_{trasl} = u_x u_x^T + u_y u_y^T$$

El resto de la comprobación la podemos hacer con codigo de la siguiente manera

```python
from typing import Tuple
import sympy as sp

Point = Tuple[sp.Symbol, sp.Symbol]

x1, x2, x3, y1, y2, y3 = sp.symbols("x_1 x_2 x_3 y_1 y_2 y_3")
k = sp.Symbol('k')
```

```python

R_1 = (-sp.sqrt(3)/2, -sp.Rational(1,2))
R_2 = ( sp.sqrt(3)/2, -sp.Rational(1,2))
R_3 = (0, 1)

def distancia(p1: Point, p2: Point) -> sp.Expr:
    return sp.sqrt((p1[0] - p2[0])**2 + (p1[1] - p2[1])**2)

def avanzar_tiempo(atomo: str = '1'):
    pos_equiv = globals()[f"R_{atomo}"]
    plus_x = globals()[f"x{atomo}"]
    plus_y = globals()[f"y{atomo}"]
    return (pos_equiv[0] + plus_x, pos_equiv[1] + plus_y)

def U_resorte(l_new):
    return sp.Rational(1, 2) * k * (l_new - sp.sqrt(3))**2

def calcular_matriz(U, derivandos):
    def doble_derivada(U, i, j):
        return sp.diff(sp.diff(U, i), j)

    n = len(derivandos)

    matrix = [[sp.Integer(0) for _ in range(n)] for _ in range(n)]

    for n_i, i in enumerate(derivandos):
        for n_j, j in enumerate(derivandos):
            derivada_segunda = doble_derivada(U, i, j)
            valor_equilibrio = derivada_segunda.subs({var: 0 for var in derivandos})
            matrix[n_i][n_j] = valor_equilibrio
    return sp.Matrix(matrix)


puntos = ['1', '2', '3']

U: sp.Expr = 0
for i in puntos:
    for j in puntos:
        if j >= i:
            continue
        new_r_i = avanzar_tiempo(i)
        new_r_j = avanzar_tiempo(j)
        dist = distancia(new_r_i, new_r_j)
        dist_simplificada = sp.simplify(dist)
        U += U_resorte(dist_simplificada)

U_simpl = sp.simplify(U)

m = sp.Symbol('m')

h = [x1, y1, x2, y2, x3, y3]

K = calcular_matriz(U, h)

M = m * sp.eye(len(h))
```

```python

Lambda = M.inv() * K

ux = sp.Matrix([1, 0, 1, 0, 1, 0]) / sp.sqrt(3)
uy = sp.Matrix([0, 1, 0, 1, 0, 1]) / sp.sqrt(3)

E_trasl = sp.simplify(ux * ux.T + uy * uy.T)

print("Matriz $E_{trasl}$:")
print(f"$${sp.latex(E_trasl)}$$")

print(f"\nTraza de $E_{{trasl}}$: ${sp.latex(E_trasl.trace())}$")

verificacion_trasl = sp.simplify(Lambda * E_trasl)
is_zero_mode = verificacion_trasl == sp.zeros(6, 6)

print(f"\n$\\Lambda * E_{{trasl}}$ es cero: {is_zero_mode}")
```

Matriz E_{trasl}:

$$\begin{bmatrix} \frac{1}{3} & 0 & \frac{1}{3} & 0 & \frac{1}{3} & 0 \\ 0 & \frac{1}{3} & 0 & \frac{1}{3} & 0 & \frac{1}{3} \\ \frac{1}{3} & 0 & \frac{1}{3} & 0 & \frac{1}{3} & 0 \\ 0 & \frac{1}{3} & 0 & \frac{1}{3} & 0 & \frac{1}{3} \\ \frac{1}{3} & 0 & \frac{1}{3} & 0 & \frac{1}{3} & 0 \\ 0 & \frac{1}{3} & 0 & \frac{1}{3} & 0 & \frac{1}{3} \end{bmatrix}$$

Traza de E_{trasl}: 2
$\Lambda * E_{trasl}$ es cero: True

11.

Siguiendo un procedimiento similar al punto 9 tenemos

$$\begin{aligned} P^{(\alpha)} &= \frac{n_\alpha}{|G|} \sum_{g \in G} \chi^{(\alpha)}(g)^* r(g) \\ &= \frac{2}{6} \sum_{g \in G} \chi^2(g)^* r(g) \end{aligned}$$

Podemos entonces calcular esto por medio de un codigo. Sin embargo, antes de hacerlo y para no repetirnos es impotante mencionar que haremos uso de las traslaciones con el proyector que ya calculamos para quitarlo de P^2 de modo que

$$P_{vib} = P^2 - E_{tral}$$

esto puesto que realmente no hay ningun cambio importante al trasladar la particula en el espacio. Luego de eso continuaremos de manera parecida a lo hecho en el punto 9.

```python
from typing import Tuple
import sympy as sp

Point = Tuple[sp.Symbol, sp.Symbol]

x1, x2, x3, y1, y2, y3 = sp.symbols("x_1 x_2 x_3 y_1 y_2 y_3")
k = sp.Symbol('k')

R_1 = (-sp.sqrt(3)/2, -sp.Rational(1,2))
R_2 = ( sp.sqrt(3)/2, -sp.Rational(1,2))
R_3 = (0, 1)

```

```python
def distancia(p1: Point, p2: Point) -> sp.Expr:
    return sp.sqrt((p1[0] - p2[0])**2 + (p1[1] - p2[1])**2)

def avanzar_tiempo(atomo: str = '1'):
    pos_equiv = globals()[f"R_{atomo}"]
    plus_x = globals()[f"x{atomo}"]
    plus_y = globals()[f"y{atomo}"]
    return (pos_equiv[0] + plus_x, pos_equiv[1] + plus_y)

def U_resorte(l_new):
    return sp.Rational(1, 2) * k * (l_new - sp.sqrt(3))**2

def calcular_matriz(U, derivandos):
    def doble_derivada(U, i, j):
        return sp.diff(sp.diff(U, i), j)

    n = len(derivandos)

    matrix = [[sp.Integer(0) for _ in range(n)] for _ in range(n)]

    for n_i, i in enumerate(derivandos):
        for n_j, j in enumerate(derivandos):
            derivada_segunda = doble_derivada(U, i, j)
            valor_equilibrio = derivada_segunda.subs({var: 0 for var in derivandos})
            matrix[n_i][n_j] = valor_equilibrio
    return sp.Matrix(matrix)

puntos = ['1', '2', '3']

U: sp.Expr = 0
for i in puntos:
    for j in puntos:
        if j >= i:
            continue
        new_r_i = avanzar_tiempo(i)
        new_r_j = avanzar_tiempo(j)
        dist = distancia(new_r_i, new_r_j)
        dist_simplificada = sp.simplify(dist)
        U += U_resorte(dist_simplificada)

U_simpl = sp.simplify(U)

m = sp.Symbol('m')

h = [x1, y1, x2, y2, x3, y3]

K = calcular_matriz(U, h)

M = m * sp.eye(len(h))

Lambda = M.inv() * K

sqrt3 = sp.sqrt(3)

D_e = sp.eye(2)
```

```python
D_rho = sp.Matrix([[-sp.S(1)/2, -sqrt3/2], [sqrt3/2, -sp.S(1)/2]])
D_tau = sp.Matrix([[-sp.S(1)/2, sqrt3/2], [-sqrt3/2, -sp.S(1)/2]])

D_s3 = sp.Matrix([[-1, 0], [0, 1]])
D_s1 = D_rho * D_s3 * D_tau
D_s2 = D_tau * D_s3 * D_rho

P_e = sp.eye(3)
P_rho = sp.Matrix([[0, 0, 1], [1, 0, 0], [0, 1, 0]])
P_tau = sp.Matrix([[0, 1, 0], [0, 0, 1], [1, 0, 0]])

P_s1 = sp.Matrix([[1, 0, 0], [0, 0, 1], [0, 1, 0]])
P_s2 = sp.Matrix([[0, 0, 1], [0, 1, 0], [1, 0, 0]])
P_s3 = sp.Matrix([[0, 1, 0], [1, 0, 0], [0, 0, 1]])

operaciones = [
    sp.kronecker_product(P_e, D_e),
    sp.kronecker_product(P_rho, D_rho),
    sp.kronecker_product(P_tau, D_tau),
    sp.kronecker_product(P_s1, D_s1),
    sp.kronecker_product(P_s2, D_s2),
    sp.kronecker_product(P_s3, D_s3)
]

chi2 = [2, -1, -1, 0, 0, 0]
P2 = (sp.S(2)/6) * sum([chi2[i] * operaciones[i] for i in range(6)], sp.zeros(6, 6))


P2_clean = sp.nsimplify(P2)
print(f"$P^2$ completo:\n $${sp.latex(P2_clean)}$$")

ux = sp.Matrix([1, 0, 1, 0, 1, 0]) / sp.sqrt(3)
uy = sp.Matrix([0, 1, 0, 1, 0, 1]) / sp.sqrt(3)

E_trasl = sp.simplify(ux * ux.T + uy * uy.T)

P_vib = sp.simplify(P2 - E_trasl)

print(f"$P_{{vib}}$: $${sp.latex(sp.simplify(P_vib))}$$")
print(f"Traza de $P_{{vib}}$: ${sp.latex(sp.simplify(P_vib.trace()))}$", end='\n\n')

v2a = sp.simplify(P_vib * sp.Matrix([1, 0, 0, 0, 0, 0]))
v2b = sp.simplify(P_vib * sp.Matrix([0, 1, 0, 0, 0, 0]))

Lambda_v2a = sp.simplify(Lambda * v2a)
idx = next(i for i in range(6) if v2a[i] != 0)
lambda_2 = sp.simplify(Lambda_v2a[idx] / v2a[idx])

print(f"Valor propio para modo E: ${sp.latex(lambda_2)}$")
```

Con resultado

P^2 completo:

$$\begin{bmatrix} \frac{2}{3} & 0 & \frac{1}{6} & -\frac{\sqrt{3}}{6} & \frac{1}{6} & \frac{\sqrt{3}}{6} \\ 0 & \frac{2}{3} & \frac{\sqrt{3}}{6} & \frac{1}{6} & -\frac{\sqrt{3}}{6} & \frac{1}{6} \\ \frac{1}{6} & \frac{\sqrt{3}}{6} & \frac{2}{3} & 0 & \frac{1}{6} & -\frac{\sqrt{3}}{6} \\ -\frac{\sqrt{3}}{6} & \frac{1}{6} & 0 & \frac{2}{3} & \frac{\sqrt{3}}{6} & \frac{1}{6} \\ \frac{1}{6} & -\frac{\sqrt{3}}{6} & \frac{1}{6} & \frac{\sqrt{3}}{6} & \frac{2}{3} & 0 \\ \frac{\sqrt{3}}{6} & \frac{1}{6} & -\frac{\sqrt{3}}{6} & \frac{1}{6} & 0 & \frac{2}{3} \end{bmatrix}$$

P_{vib}:

$$\begin{bmatrix} \frac{1}{3} & 0 & -\frac{1}{6} & -\frac{\sqrt{3}}{6} & -\frac{1}{6} & \frac{\sqrt{3}}{6} \\ 0 & \frac{1}{3} & \frac{\sqrt{3}}{6} & -\frac{1}{6} & -\frac{\sqrt{3}}{6} & -\frac{1}{6} \\ -\frac{1}{6} & \frac{\sqrt{3}}{6} & \frac{1}{3} & 0 & -\frac{1}{6} & -\frac{\sqrt{3}}{6} \\ -\frac{\sqrt{3}}{6} & -\frac{1}{6} & 0 & \frac{1}{3} & \frac{\sqrt{3}}{6} & -\frac{1}{6} \\ -\frac{1}{6} & -\frac{\sqrt{3}}{6} & -\frac{1}{6} & \frac{\sqrt{3}}{6} & \frac{1}{3} & 0 \\ \frac{\sqrt{3}}{6} & -\frac{1}{6} & -\frac{\sqrt{3}}{6} & -\frac{1}{6} & 0 & \frac{1}{3} \end{bmatrix}$$

Traza de P_{vib}: 2
Valor propio para modo E: $\frac{3k}{2m}$
Esto implica que tiene una frecuencia

$$\sqrt{\frac{3k}{2m}}$$

12.

12.1. $EP = PE = E$.

12.1.1. $PE = E$. Sabemos que

$$\forall v \in \mathcal{H} \implies Ev \in \Im(E) \subset \Im(P)$$

Ahora bien, las proyecciónes se comportan como la identidad en los elementos que estan dentro de su imagen (pues en caso contrario podrian cambiar al aplicarse dos veces). Por lo tanto $PEv = IEv = Ev$

12.1.2. $EP = E$. Para este caso podemos simplemente darnos cuenta que:

$$\begin{aligned} EP &= E \\ \implies (EP)^* &= E^* \\ \implies P^*E^* &= E^* \end{aligned}$$

y dado que esta es una proyección esto implica

$$\implies PE = E$$

Ya demostramos anteriormente esto.

12.2. $(P - E)$ **es una proyección.**

12.2.1. $(P - E)^* = (P - E)$. En este caso simplemente se puede aprovechar las caracteristicas de $(^*)$ y que P y E son proyecciónes (es decir $P^* = P \wedge E^* = E$)

$$\begin{aligned} (P - E)^* &= P^* - E^* \\ (P - E)^* &= P - E \end{aligned}$$

12.2.2. $(P-E)^2 = (P-E)$. De modo similar podemos desarrollar

$$\begin{aligned}
(P-E)^2 &= (P-E)(P-E) \\
&= P^2 - PE - EP + E^2 \\
P^2 &= P \\
PE &= E \\
EP &= E \\
E^2 &= E \\
&= P - E - E + E \\
&= P - E
\end{aligned}$$

13. Anexos

13.1. **Caracteres representación vectorial.**

- Identidad:

$$\begin{aligned}
tr(D^{(2)}(e)) &= tr \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \\
&= 2
\end{aligned}$$

- Rotación:

$$\begin{aligned}
tr(D^{(2)}(\tau)) &= tr \frac{1}{2} \begin{pmatrix} -1 & -\sqrt{3} \\ \sqrt{3} & -1 \end{pmatrix} \\
&= -1 \\
tr(D^{(2)}(\rho)) &= tr \frac{1}{2} \begin{pmatrix} -1 & \sqrt{3} \\ -\sqrt{3} & -1 \end{pmatrix} \\
&= -1
\end{aligned}$$

- Reflexión:

$$\begin{aligned}
tr(D^{(2)}(\sigma_1)) &= tr \frac{1}{2} \begin{pmatrix} 1 & \sqrt{3} \\ \sqrt{3} & -1 \end{pmatrix} \\
&= 0 \\
tr(D^{(2)}(\rho)) &= tr \frac{1}{2} \begin{pmatrix} 1 & -\sqrt{3} \\ -\sqrt{3} & -1 \end{pmatrix} \\
&= 0 \\
tr(D^{(2)}(\rho)) &= tr \frac{1}{2} \begin{pmatrix} -1 & 0 \\ 0 & 1 \end{pmatrix} \\
&= 0
\end{aligned}$$